جامعة أم القرى
UMM AL-QURA UNIVERSITY

Project — Knowledge Representation and Inference

College of Computing Ontology

An OWL ontology developed in Protégé

Group Members:

Reuf Bakr Alshrif
ID: 443002428

Jana Hesham Ahlis
ID: 443002347

Reham Faisal Alsubhi
ID: 444003014

Layan Adel Babkour
ID: 444002368

Manar Ali Alsubhi
ID: 444003523

Instructor: Dr. Salwa Althobaity

Course: DS2203 - Knowledge Representation and Inference

Department: Data Science Department
College of Computing
Umm Al-Qura University

June 1, 2026

Contents

1 Introduction **5**

2 TBox - Class Hierarchy **6**
2.1 Class Structure 6

3 Object Properties **7**
3.1 Property Definitions 7
3.2 Property Characteristics 7
3.2.1 Inverse Properties 7
3.2.2 Functional Properties 7

4 Data Properties **9**
4.1 Disjointness Declaration 10
4.2 Cardinality Restrictions 10
4.3 Class Equivalence (Defined Classes) 11

5 Individuals (ABox) **12**
5.1 Individual Distribution 12
5.2 Individual Examples 13
5.3 Individual Assertions 13

6 Reasoning **14**
6.1 Inferred Class Memberships 15
6.1.1 ActiveStudent Inference 15
6.1.2 DepartmentWithPrograms Inference 15
6.2 Inferred Property Relationships 15
6.2.1 Student Enrollment Inferences 15
6.2.2 Program-Department Inferences 15
6.3 Reasoning Benefits 16
6.4 Complete Query Catalog 16
6.4.1 Query 1: Basic Student Information 16
6.4.2 Query 2: Complete Student Profiles 16
6.4.3 Query 3: Committee Analytics 16
6.4.4 Query 4: Student Email Addresses 16
6.4.5 Query 5: Courses Filtered by Credit Hours 16
6.4.6 Query 6: Student-Course-Club Mapping 17
6.4.7 Query 7: Events with Participants 17
6.4.8 Query 8: Program-Specific Courses 17
6.4.9 Query 9: Course Enrollment Statistics 17
6.4.10 Query 10: Department-Program Hierarchy 17
6.4.11 Query 11: Full Organizational Hierarchy 17
6.4.12 Query 12: Comprehensive Student Data 17
6.4.13 Query 13: Student Activity Analytics 18
6.4.14 Query 14: Basic Member Profiles 18
6.5 Featured Query Results 18

6.6 Query Features Summary 20

7 Conclusion **21**
7.1 Project Summary 21
7.2 Key Achievements 22

8 Discussion **23**
8.1 Conceptual Understanding of Ontology Modelling 23
8.2 Understanding of Knowledge Representation and Inference Formalisms . . 23
8.3 Query Understanding 23

9 References **24**

10 Appendix **25**
10.1 Project Files Repository 25

List of Figures

1 Complete Class Hierarchy 6
2 Object Property Characteristics and Examples 8
3 Data Property Examples with Different Datatypes 9
4 Disjointness Declaration in Protégé 10
5 Cardinality Restriction Examples 11
6 Defined Classes with Necessary and Sufficient Conditions 12
7 Complete Individual Example: Student Profile with Property Assertions . 14
8 Query Result: Complete Student Profiles with All Attributes 18
9 Query Result: Events with Student Participants 18
10 Query Result: Department-Program Organizational Hierarchy 19
11 Query Result: Student Activity Analytics with Engagement Metrics 19

List of Tables

1 Object Properties with Domains and Ranges 7
2 Detailed Description of Object Property Screenshots 8
3 Data Properties with Datatypes 9
4 Consistency Verification Results 14
5 Ontology Project Components Summary 21
6 Project Files Access 25

1 Introduction

This report presents a comprehensive OWL ontology developed in Protégé that models the organizational structure, academic activities, and administrative components of the College of Computing at Umm Al-Qura University. The ontology captures entities such as departments, programs, courses, faculty, students, committees, facilities, and events, along with their relationships and constraints. The ontology was designed using publicly available information from the college website (`https://uqu.edu.sa/en/ccomp`) and adheres to best practices in ontology engineering, including proper class hierarchies, property definitions, axioms, and reasoning capabilities.

2 TBox - Class Hierarchy

2.1 Class Structure

The ontology defines 10 domain-relevant classes organized into a logical hierarchy using subclass relationships:

- **College** - Represents the college as a whole
- **Department** - Represents academic departments within the college
- **Program** - Represents academic programs offered by departments
- **Course** - Represents individual courses within programs
- **Person** - Abstract class representing people associated with the college
  - **Student** - Subclass of Person representing enrolled students
  - **Faculty** - Subclass of Person representing teaching staff
- **Event** - Represents college events and activities
- **Committee** - Represents student clubs and committees
- **Facility** - Represents physical facilities and infrastructure

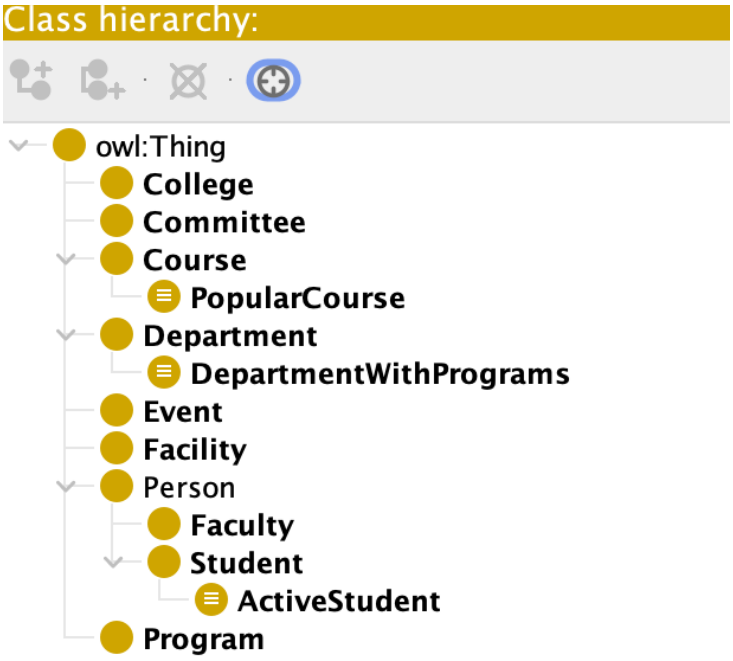


Figure 1: Complete Class Hierarchy

The hierarchy demonstrates proper use of inheritance, with Student and Faculty as specialized subclasses of the more general Person class.

3 Object Properties

3.1 Property Definitions

The ontology includes 12 object properties that describe relationships between classes:

Property	Domain	Range
hasDepartment	College	Department
offersProgram	Department	Program
offersCourse	Program	Course
enrolledIn	Student	Course
hasStudent	Course	Student
teaches	Faculty	Course
taughtBy	Course	Faculty
memberOf	Person	Committee
participatesIn	Person	Event
locatedIn	Facility	College
belongsToDepartment	Program	Department
belongsToProgram	Course	Program

Table 1: Object Properties with Domains and Ranges

3.2 Property Characteristics

The ontology utilizes the following property characteristics:

3.2.1 Inverse Properties

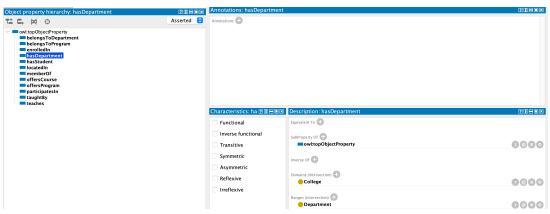
The following inverse property pairs were defined:

- enrolledIn $\leftrightarrow$ hasStudent
- teaches $\leftrightarrow$ taughtBy
- offersProgram $\leftrightarrow$ belongsToDepartment
- offersCourse $\leftrightarrow$ belongsToProgram

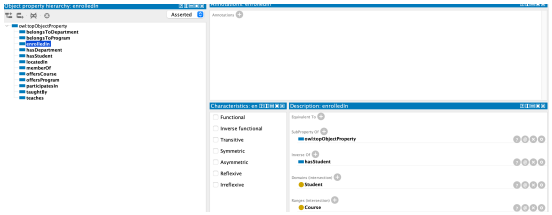
3.2.2 Functional Properties

The following properties were declared as functional:

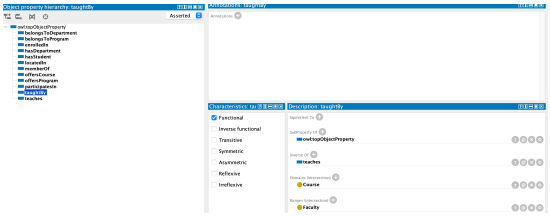
- **taughtBy** - Each course is taught by one specific faculty member
- **locatedIn** - Each facility is located in one specific college



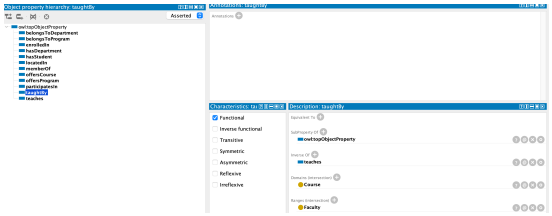
(a) Domain and Range



(b) Inverse Property



(c) Functional Property



(d) Property Example

Figure 2: Object Property Characteristics and Examples

Figure	**Description**
(a)	The hasDepartment property with its domain defined as College and range as Department, demonstrating proper type constraints
(b)	The inverse relationship between enrolledIn and hasStudent, enabling bidirectional navigation in the ontology
(c)	The taughtBy property marked as functional, ensuring each course is taught by exactly one faculty member
(d)	Another object property example showing complete property definition with all characteristics

Table 2: Detailed Description of Object Property Screenshots

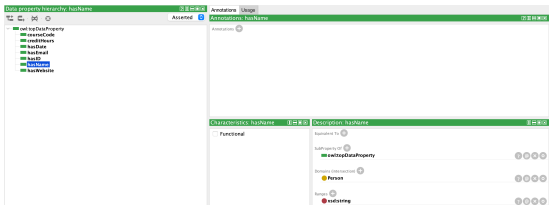
4 Data Properties

Data properties are used to describe literal attributes of individuals in the ontology. Unlike object properties which relate individuals to other individuals, data properties associate individuals with data values such as strings, integers, dates, and URIs. The ontology defines 7 data properties with appropriate datatypes to capture essential information about students, courses, events, and the college itself. Each data property has a well-defined domain (the class it applies to) and range (the datatype of its values). This ensures type safety and consistency across the ontology.

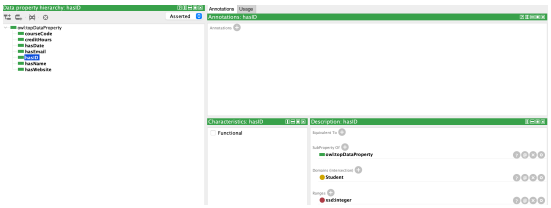
The following table summarizes all data properties defined in this ontology:

Property	Domain	Range
hasName	Person	xsd:string
hasEmail	Person	xsd:string
hasID	Student	xsd:integer
courseCode	Course	xsd:string
creditHours	Course	xsd:integer
hasWebsite	College	xsd:string
hasDate	Event	xsd:dateTime

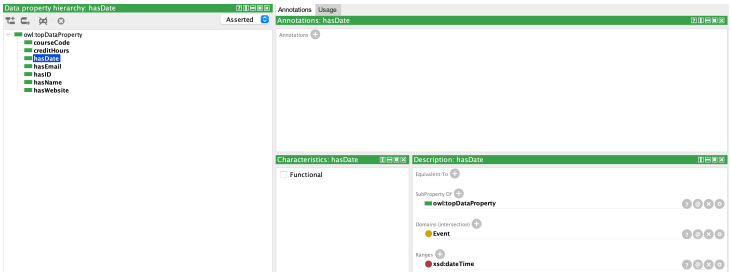
Table 3: Data Properties with Datatypes



(a) String Data Property (hasName)



(b) Integer Data Property (hasID)



(c) DateTime Data Property (hasDate)

Figure 3: Data Property Examples with Different Datatypes

These properties use a variety of datatypes including string, integer, and dateTime, providing rich descriptive attributes for individuals in the ontology.

4.1 Disjointness Declaration

Disjointness axioms prevent logical contradictions by ensuring that individuals cannot simultaneously belong to mutually exclusive classes.
The following disjointness was declared:

- **Student DisjointWith Faculty**

This ensures that no individual can be both a student and a faculty member.

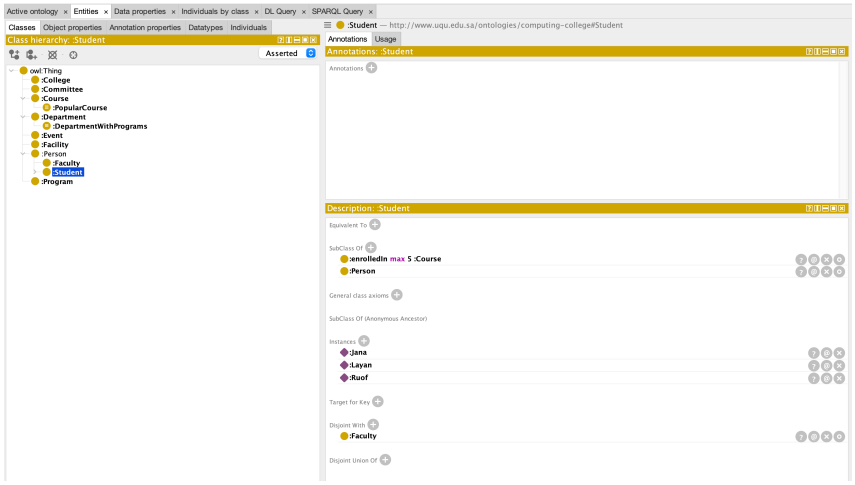


Figure 4: Disjointness Declaration in Protégé

4.2 Cardinality Restrictions

Cardinality restrictions are logical constraints that specify the minimum or maximum number of relationships an individual can have with other individuals. These restrictions enforce business rules and domain knowledge directly in the ontology structure, ensuring that certain requirements are always met.

Three cardinality restrictions were implemented to ensure logical constraints:

1. **Department must offer at least 1 Program:**

   $\texttt{Department} \sqsubseteq \texttt{offersProgram min 1 Program}$

   This ensures that every academic department offers at least one program, preventing the existence of empty departments.

2. **Program must offer at least 1 Course:**

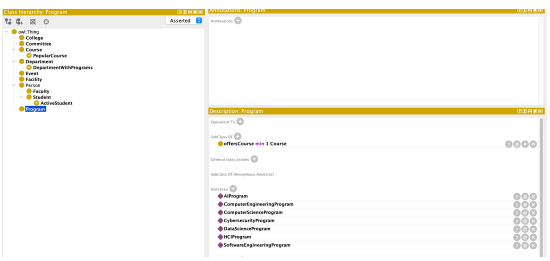
   $\texttt{Program} \sqsubseteq \texttt{offersCourse min 1 Course}$

   This ensures that every academic program offers at least one course, preventing the existence of empty programs.

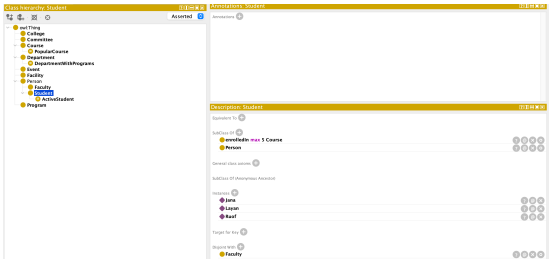
3. **Student can enroll in maximum 5 Courses:**

   $\texttt{Student} \sqsubseteq \texttt{enrolledIn max 5 Course}$

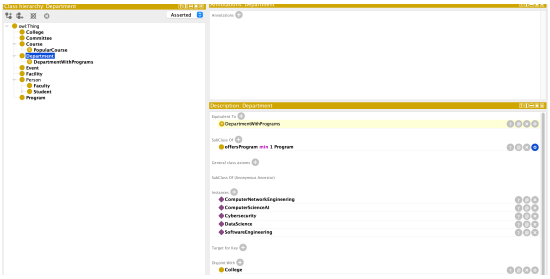
   This limits course enrollment to maintain a reasonable academic load per student.



(a) Program: min 1 course



(b) Student: max 5 courses



(c) Department: min 1 program

Figure 5: Cardinality Restriction Examples

4.3 Class Equivalence (Defined Classes)

Class equivalence, also known as defined classes, uses necessary and sufficient conditions to specify exactly which individuals belong to a class. Unlike primitive classes (defined only by necessary conditions), defined classes allow the reasoner to automatically infer class membership based on an individual's properties and relationships.
When an individual satisfies the necessary and sufficient conditions of a defined class, the reasoner automatically classifies that individual as a member of the class. This enables dynamic classification without requiring explicit assertion of class membership.

Three equivalence classes were defined to identify key entity types:

1. **ActiveStudent** - A student enrolled in at least one course:

   `ActiveStudent` ≡ `Student` ⊓ `(enrolledIn some Course)`

   Any student who is enrolled in at least one course will be automatically classified as an ActiveStudent by the reasoner.

2. **PopularCourse** - A course with at least 2 enrolled students:

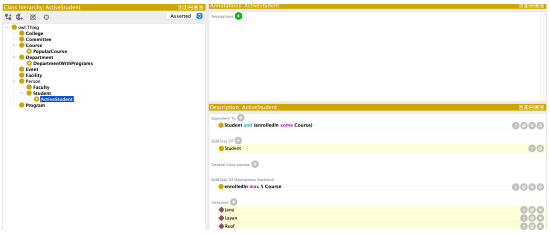
   `PopularCourse` ≡ `Course` ⊓ `(hasStudent min 2 Student)`

   Courses with two or more enrolled students are automatically identified as popular courses.

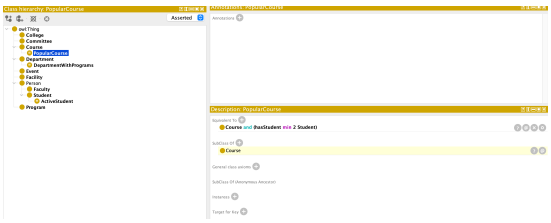
3. **DepartmentWithPrograms** - A department offering at least one program:

   `DepartmentWithPrograms` ≡ `Department` ⊓ `(offersProgram some Program)`

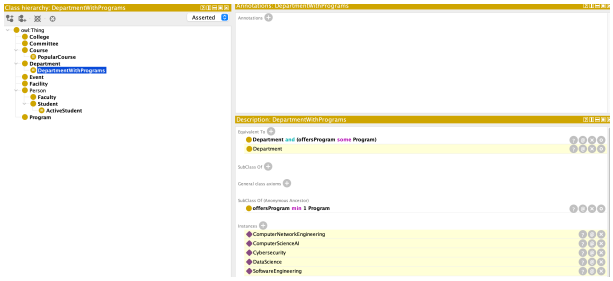
   Departments that offer at least one academic program are automatically classified as active departments.



(a) ActiveStudent



(b) PopularCourse



(c) DepartmentWithPrograms

Figure 6: Defined Classes with Necessary and Sufficient Conditions

These definitions enable the reasoner to automatically infer class memberships based on individual properties and relationships.

5 Individuals (ABox)

The ABox (Assertional Box) contains concrete instances of the classes defined in the TBox. These individuals represent real-world entities from the College of Computing at Umm Al-Qura University, populated with actual data and relationships.
This ontology includes 25 individuals distributed across all main classes, ensuring comprehensive coverage of the domain. Each individual is properly instantiated with class membership, object property assertions linking it to other individuals, and data property assertions providing descriptive attributes.

5.1 Individual Distribution

The ontology includes **25 individuals** distributed across the following classes:

- **College:** 1 individual
- **Department:** 5 individuals
- **Program:** 7 individuals
- **Course:** 3 individuals
- **Student:** 3 individuals
- **Committee:** 3 individuals
- **Event:** 3 individuals

5.2 Individual Examples

Key individuals include:

- **CollegeOfComputing** - The main college entity with associated departments
- **Students:** Ruof, Jana, and Layan - with complete profiles including IDs and emails
- **Courses:** CS101, CS102, CS103 - with course codes and credit hours
- **Departments:** Five departments covering different computing disciplines
- **Programs:** Seven academic programs including Computer Science, AI, and Cybersecurity

5.3 Individual Assertions

Each individual includes comprehensive assertions:

- **Class Membership** - Explicit type declaration (e.g., Ruof is a Student)
- **Object Property Assertions** - Relationships with other individuals:
  - CollegeOfComputing hasDepartment ComputerScienceAI
  - Ruof enrolledIn CS101
  - Jana memberOf InnovationClub
- **Data Property Assertions** - Literal attribute values:
  - Ruof hasName "Ruof"
  - Ruof hasID 443002428
  - Ruof hasEmail "s443002428@uqu.edu.sa"
  - CS101 courseCode "CS101"
  - CS101 creditHours 3

Figure 7 demonstrates a complete individual with all property assertions including relationships and attributes.

These individuals, along with their property assertions, form a complete knowledge base that accurately represents the structure and relationships within the College of Computing.

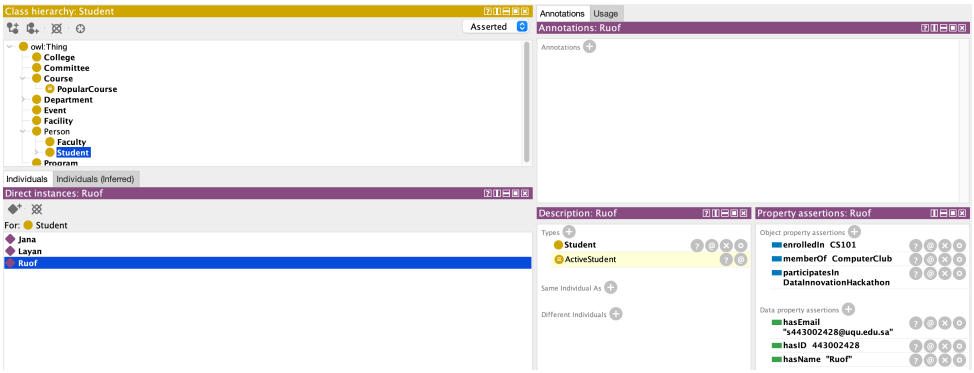


Figure 7: Complete Individual Example: Student Profile with Property Assertions

6 Reasoning

Reasoning is a critical step in ontology development that validates logical consistency and derives implicit knowledge from explicit assertions. An OWL reasoner uses the defined axioms, properties, and class definitions to infer new facts that are logically entailed but not explicitly stated.
This ontology was tested using the HermiT reasoner to verify consistency and demonstrate automated inference capabilities.

The ontology was verified using the HermiT reasoner and confirmed to be logically consistent. Consistency checking ensures that the ontology contains no contradictions that would make it unusable for inference.

The successful consistency check validates the following aspects:

Validation Aspect	**Description**
No Logical Contradictions	All axioms and constraints are compatible with each other
Satisfiable Class Definitions	Every class can have at least one valid instance
Valid Individual Assertions	Individual data conforms to class constraints and restrictions
Disjointness Compliance	No individual belongs to mutually exclusive classes
Cardinality Satisfaction	All minimum and maximum relationship constraints are met

Table 4: Consistency Verification Results

This comprehensive validation confirms that the ontology is logically sound and ready for automated reasoning and inference.

6.1 Inferred Class Memberships

One of the key benefits of defined classes with necessary and sufficient conditions is automatic classification. The reasoner automatically assigns individuals to defined classes based on their properties and relationships, without requiring explicit class membership assertions.

6.1.1 ActiveStudent Inference

All three students (Ruof, Jana, Layan) were automatically classified as members of the ActiveStudent class. This inference occurred because each student is enrolled in at least one course, which satisfies the equivalence definition:
`ActiveStudent` $\equiv$ `Student` $\sqcap$ `(enrolledIn some Course)`
The reasoner analyzed each student's enrolledIn property and automatically added them to the ActiveStudent class without any manual assertion.

6.1.2 DepartmentWithPrograms Inference

Departments that offer at least one academic program were automatically classified as members of the DepartmentWithPrograms class. The reasoner evaluated each department's offersProgram relationships and inferred membership based on the defined equivalence conditions. This demonstrates how defined classes enable dynamic classification as the ontology evolves.

6.2 Inferred Property Relationships

The reasoner successfully derived implicit relationships using inverse property definitions. When an object property assertion is made in one direction, its inverse relationship is automatically inferred in the opposite direction without requiring explicit assertion.

6.2.1 Student Enrollment Inferences

When students were asserted to be enrolled in courses, the reasoner automatically inferred the inverse relationships. For example, when Ruof was asserted as enrolledIn CS101, the reasoner automatically inferred that CS101 hasStudent Ruof. Similarly, Jana enrolledIn CS102 resulted in the inference that CS102 hasStudent Jana, and Layan enrolledIn CS103 led to CS103 hasStudent Layan.

6.2.2 Program-Department Inferences

The inverse property pairs between programs and departments also generated automatic inferences. When ComputerScienceAI was asserted to offersProgram AIProgram, the reasoner automatically inferred that AIProgram belongsToDepartment ComputerScienceAI. This bidirectional relationship inference applies to all program-department connections in the ontology.

6.3 Reasoning Benefits

These automatic inferences demonstrate the ontology's logical coherence and the power of semantic reasoning in OWL. The reasoner successfully validated all axioms and derived new knowledge without requiring explicit assertions. This capability significantly reduces manual data entry, eliminates redundant assertions, and maintains logical consistency throughout the ontology. The inferred knowledge is as reliable as explicitly stated facts, providing a complete and coherent knowledge base.

6.4 Complete Query Catalog

The 14 queries executed demonstrate the full spectrum of SPARQL capabilities available for ontology querying and data analysis. Each query serves a specific purpose while showcasing different SPARQL features and techniques.

6.4.1 Query 1: Basic Student Information

The first query retrieves basic student information including names and IDs using simple SELECT statements. This demonstrates fundamental SPARQL syntax and the ability to extract data properties from individuals of a specific class.

6.4.2 Query 2: Complete Student Profiles

The second query retrieves complete student profiles with comprehensive information. This advanced query combines multiple data properties such as names and emails with object properties including course enrollments, club memberships, and event participation. The query uses OPTIONAL clauses to handle incomplete data gracefully, ensuring that students without certain relationships are still included in results.

6.4.3 Query 3: Committee Analytics

The third query performs committee analytics by grouping students by their committee memberships and counting members per committee. This demonstrates SPARQL's aggregation capabilities using GROUP BY and COUNT functions to transform raw relationship data into meaningful statistical insights.

6.4.4 Query 4: Student Email Addresses

The fourth query simply retrieves student email addresses, demonstrating basic data property access. While simple, this query shows how SPARQL can efficiently extract specific attributes from individuals without requiring complex patterns.

6.4.5 Query 5: Courses Filtered by Credit Hours

The fifth query filters courses by credit hours, retrieving only courses with exactly 3 credit hours. This demonstrates the FILTER clause capability to apply conditions on data property values, enabling selective data retrieval based on specific criteria.

6.4.6 Query 6: Student-Course-Club Mapping

The sixth query creates a mapping between students, their enrolled courses, and their club memberships. This query uses BIND for string manipulation and DISTINCT to ensure unique results, demonstrating how SPARQL can combine multiple relationships and format output for better readability.

6.4.7 Query 7: Events with Participants

The seventh query retrieves events along with their student participants. It demonstrates string manipulation capabilities using BIND and REPLACE functions to extract readable event names from URIs, making query results more user-friendly.

6.4.8 Query 8: Program-Specific Courses

The eighth query navigates through the program-course relationship to find all courses offered by a specific program. This demonstrates targeted navigation through the ontology structure with filtering based on named individuals.

6.4.9 Query 9: Course Enrollment Statistics

The ninth query counts the number of students enrolled in each course, providing enrollment statistics. This simple aggregation query uses COUNT with GROUP BY to analyze course popularity across the student body.

6.4.10 Query 10: Department-Program Hierarchy

The tenth query performs multi-level hierarchical navigation from the college through departments to programs. This demonstrates SPARQL's ability to traverse complex organizational structures and extract relationships spanning multiple levels of the class hierarchy.

6.4.11 Query 11: Full Organizational Hierarchy

The eleventh query extends hierarchical navigation to four levels, traversing from college through departments and programs to individual courses. This complex query shows how SPARQL can navigate deep relationship chains while formatting output using BIND operations.

6.4.12 Query 12: Comprehensive Student Data

The twelfth query retrieves comprehensive student profiles including all available information about courses, clubs, and events. Using multiple OPTIONAL clauses, this query ensures complete data retrieval while handling missing relationships gracefully.

6.4.13 Query 13: Student Activity Analytics

The thirteenth query performs advanced student activity analytics by simultaneously counting distinct courses, clubs, and events for each student. This sophisticated aggregation query uses multiple COUNT DISTINCT operations with GROUP BY to provide comprehensive engagement metrics, ranked by event participation.

6.4.14 Query 14: Basic Member Profiles

The fourteenth query retrieves basic student profiles focused on membership tracking, extracting fundamental information about students and their organizational affiliations. This query balances simplicity with usefulness for common reporting needs.

6.5 Featured Query Results

Four representative queries were selected to showcase the most important SPARQL features through visual results. These queries demonstrate essential capabilities required for comprehensive ontology interaction and data analysis.

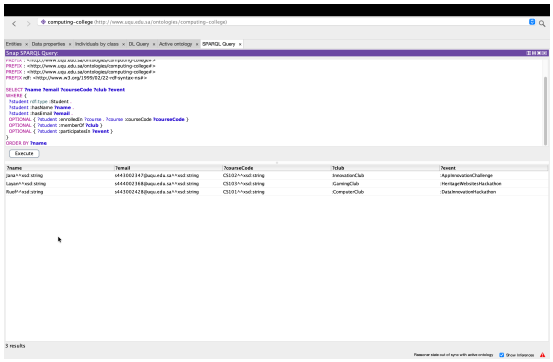


Figure 8: Query Result: Complete Student Profiles with All Attributes

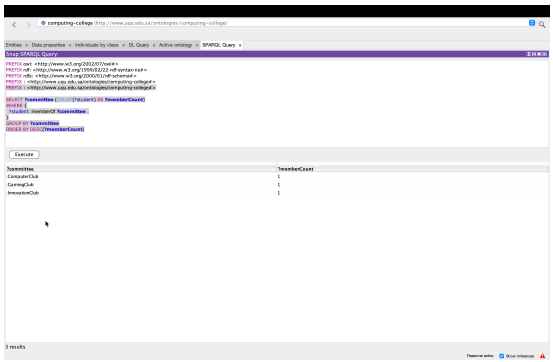


Figure 9: Query Result: Events with Student Participants

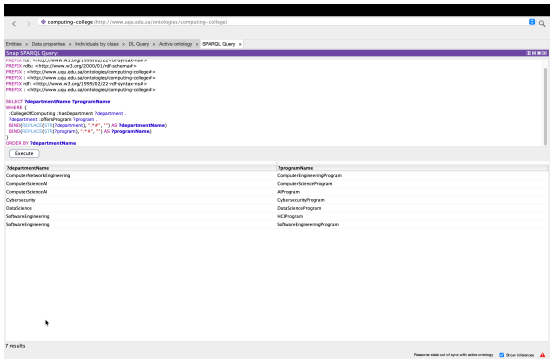


Figure 10: Query Result: Department-Program Organizational Hierarchy

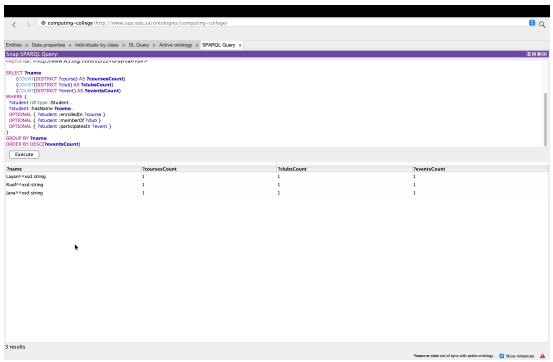


Figure 11: Query Result: Student Activity Analytics with Engagement Metrics

6.6 Query Features Summary

Basic Retrieval (Queries 1-4)

1. **Basic Student Info:** SELECT, data properties, class filtering
2. **Complete Profiles:** OPTIONAL, multiple properties, ORDER BY
3. **Committee Analytics:** GROUP BY, COUNT, aggregation
4. **Student Emails:** Basic data property access

Filtering & Manipulation (Queries 5-8)

5. **Courses by Credits:** FILTER, conditional retrieval
6. **Student-Course-Club:** BIND, DISTINCT, string manipulation
7. **Events Participants:** BIND, REPLACE, URI formatting
8. **Program Courses:** Relationship navigation, filtering

Hierarchical Navigation (Queries 9-11)

9. **Enrollment Stats:** COUNT, GROUP BY
10. **Dept-Program Tree:** Multi-level hierarchical navigation
11. **Full Hierarchy:** Four-level navigation, BIND

Complex Aggregation (Queries 12-14)

12. **All Student Data:** Multiple OPTIONAL clauses
13. **Activity Metrics:** COUNT DISTINCT, ORDER BY DESC
14. **Member Profiles:** Basic tracking, reporting

7 Conclusion

This project successfully demonstrated the complete lifecycle of ontology development through the creation of a comprehensive OWL ontology modeling the College of Computing at Umm Al-Qura University.

7.1 Project Summary

The following table summarizes the key components and achievements of this ontology project:

Component	**Implementation Details**
Class Hierarchy (TBox)	10 classes including College, Department, Program, Course, Person (with Student and Faculty subclasses), Committee, Event, and Facility
Object Properties	12 properties with domains, ranges, inverse relationships, and functional characteristics
Data Properties	7 properties with appropriate datatypes (string, integer, dateTime)
Axioms & Constraints	3 cardinality restrictions, disjointness declarations, and 3 defined classes with necessary and sufficient conditions
Individuals (ABox)	25 fully instantiated individuals across all main classes
Reasoning	HermiT reasoner confirmed consistency and performed automatic classification and property inference
SPARQL Queries	14 diverse queries demonstrating pattern matching, filtering, aggregation, and hierarchical navigation

Table 5: Ontology Project Components Summary

7.2 Key Achievements

The ontology successfully captures essential entities, relationships, and constraints defining the academic structure of a computing college. The TBox established a well-structured class hierarchy while object and data properties enabled rich knowledge representation with bidirectional navigation.

Domain-specific business rules were enforced through cardinality restrictions, disjointness declarations, and defined classes enabling automatic classification. The HermiT reasoner validated logical consistency and demonstrated automatic inference by classifying individuals into defined classes and inferring inverse property relationships. SPARQL querying was extensively validated through 14 queries ranging from simple

retrieval to complex analytics, demonstrating comprehensive data access capabilities including pattern matching, optional patterns, filtering, aggregation, and hierarchical navigation. This ontology serves as a functional knowledge base for academic

management systems, demonstrating mastery of semantic web technologies and illustrating how formal ontologies capture complex domain knowledge in machine interpretable format while enabling automated reasoning and flexible querying.

8 Discussion

8.1 Conceptual Understanding of Ontology Modelling

The ontology represents the real-world structure of the College of Computing as a formal knowledge model. The class hierarchy organizes the main academic entities, including `College`, `Department`, `Program`, `Course`, `Person`, `Student`, `Faculty`, `Event`, `Committee`, and `Facility`. The use of `Person` as a parent class for `Student` and `Faculty` demonstrates inheritance and specialization. Shared properties such as `hasName` and `hasEmail` are defined at the parent level, while subclass-specific properties remain flexible. Object properties reflect real academic relationships. For example, `hasDepartment`, `offersProgram`, and `offersCourse` represent the academic structure, while `enrolledIn`, `teaches`, and `memberOf` model interactions between individuals and entities. Inverse properties such as `enrolledIn` and `hasStudent` support bidirectional navigation.

8.2 Understanding of Knowledge Representation and Inference Formalisms

The ontology applies OWL constructs to support knowledge representation and automated reasoning. Cardinality restrictions enforce logical constraints, such as requiring departments to offer at least one program and limiting student enrollment. Disjointness axioms prevent logical contradictions. For example, `Student` and `Faculty` are disjoint classes, ensuring that one individual cannot belong to both classes simultaneously. Defined classes such as `ActiveStudent`, `PopularCourse`, and `DepartmentWithPrograms` demonstrate automatic inference. For instance, when a student is enrolled in a course, the reasoner can automatically classify the individual as an `ActiveStudent`.

8.3 Query Understanding

SPARQL queries were used to retrieve and analyze ontology knowledge. Basic queries extracted information about students, courses, committees, and events, while relationship queries identified enrollments, memberships, and participation links. The use of `OPTIONAL` clauses enabled flexible retrieval even when some relationships were missing. Aggregation functions such as `COUNT` were also used to generate summary statistics and analytical results. These queries demonstrate that the ontology supports both simple information retrieval and advanced knowledge analysis.

9 References

1. Protégé Ontology Editor. Stanford Center for Biomedical Informatics Research. https://protege.stanford.edu/

2. OWL 2 Web Ontology Language Primer (Second Edition). W3C Recommendation, 11 December 2012. https://www.w3.org/TR/owl2-primer/

3. SPARQL 1.1 Query Language. W3C Recommendation, 21 March 2013. https://www.w3.org/TR/sparql11-query/

4. Shearer, R., Motik, B., & Horrocks, I. (2008). HermiT: A Highly-Efficient OWL Reasoner. OWLED.

10 Appendix

10.1 Project Files Repository

Resource	Access Information
Screenshots Archive	`https://drive.google.com/drive/folders/1VdAZBGfUZkGjso2lp7mOlkP_BXbWpR1h?usp=sharing`
Ontology File	`College_Of_Computing__Ontologe.owx` Format: OWL/XML (.owx)

Table 6: Project Files Access